Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Інститут комп’ютерних наук та інформаційних технологій

Кафедра САП



Методичні вказівки до лабораторної роботи № 5

з дисципліни: “Організація баз даних та знань”
на тему:
“**Написання запитів SELECT. Ч. 2**”

Затверджено на засіданні кафедри
“*Системи автоматизованого проєктування*”
Протокол № 1 від 25.08.2026 р.

Львів-2026 р.

НАПИСАННЯ ЗАПИТІВ SELECT. Ч. 2. Методичні вказівки до лабораторної роботи № 5 з курсу “Організація баз даних та знань” для студентів базового напрямку “Комп’ютерні науки” / Укл. О. М. Станкевич. – Львів: НУЛП, 2026. – 39 с.

Укладач: О. М. Станкевич

1. МЕТА РОБОТИ

Мета роботи – отримання практичних навичок об’єднання даних, написання аналітичних та підсумкових запитів, виконання теоретико-множинних операцій реляційної алгебри засобами SQL.

2. ТЕОРЕТИЧНІ ВІДОМОСТІ

Підзапити – це вирази **SELECT**, вбудовані в інші запити SQL. Розглянемо простий приклад застосування підзапитів. Наприклад, створимо таблицю для товарів і замовлень:

```
4  create table products(
5  id int auto_increment primary key,
6  product_name varchar(30) not null,
7  manufacturer varchar(30) not null,
8  product_count int default 0,
9  price decimal not null);
10
11 create table orders(
12 id int auto_increment primary key,
13 product_id int not null,
14 product_count int default 1,
15 created_at date not null,
16 price decimal not null,
17 foreign key (product_id) references products(id) on delete cascade
18 );
```

Таблиця orders містить дані про реалізований товар із таблиці products. Додамо в таблиці деякі дані:

```
insert into products (product_name, manufacturer, product_count, price)
values
('iphone X', 'Apple', 2, 76000),
('iphone 8', 'Apple', 2, 51000),
('iphone 7', 'Apple', 5, 42000),
('Galaxy S9', 'Samsung', 2, 56000),
('Galaxy 8', 'Samsung', 1, 46000),
('Honor 10', 'Huawei', 2, 26000),
('Nokia 8', 'HMD Global', 6, 38000);
```

```
insert into orders (product_id, created_at, product_count,  price)
values (
(select id from products where product_name='Galaxy S8'),
'2018-05-21',
2,
(select price from products where product_name='Galaxy S8')
),
(
(select id from products where product_name='iphone X'),
'2018-05-23',
1,
(select price from products where product_name='iphone X')
),
(
(select id from products where product_name='iphone 8'),
'2018-05-21',
1,
(select price from products where product_name='iphone 8')
);
```

Для додавання даних в таблицю orders використали підзапити. Наприклад, перше замовлення зроблено на товар Galaxy S8. Відповідно до таблиці orders нам треба додати інформацію про замовлення, де поле product_id вказує на id товару Galaxy S8, поле price – на його ціну. Але на момент написання запиту нам може бути невідомий ні id покупця, ні id товару, ні ціна товару. У цьому випадку можна виконати підзапит в вигляді:

```
(select id from products where product_name='iphone 8'),
```

Підзапит виконує команду **SELECT** і береться в дужки. У цьому ж випадку при додаванні одного товару виконується два підзапити. Кожен підзапит повертає одне скалярне значення, наприклад, числовий ідентифікатор.

У наведеному вище прикладі підзапити виконувалися до іншої таблиці, але можуть виконуватися і до тієї ж, для якої викликається основний запит. Наприклад, знайдемо товари з таблиці products, які мають мінімальну ціну:

```
53 •  select * from products
54    where price = (select min(price) from products);
55
```

Result Grid | Filter Rows: | Edit: | Export/Import:

id	product_name	manufacturer	product_count	price
6	Honor 10	Huawei	2	26000
NULL	NULL	NULL	NULL	NULL

Корелюючі та некорелюючі підзапити.

Підзапити бувають корелюючі та некорелюючі. У наведених вище прикладах команди **SELECT** фактично виконували один підзапит для всіх рядків. Наприклад, підзапит повертає мінімальну або середню ціну, яка не зміниться, скільки б ми рядків не вибирали в основному запиті. Тобто, результат підзапиту не залежав від рядків, які у ньому вибиралися. І такий підзапит виконується один раз для всього зовнішнього запиту. Наведені вище приклади належать до *некорелюючих підзапитів*.

Але також можна використовувати і *корелюючі підзапити* (correlated subquery), результати яких залежать від рядків, які вибираються в основному запиті.

Наприклад, виберемо всі замовлення з таблиці orders, додавши до них інформацію про товар:

```
53 •  select created_at, price,
54        (select product_name from products
55         where products.id = orders.product_id) as product
56    from orders;
57
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

created_at	price	product
2018-05-21	46000	Galaxy S8
2018-05-23	76000	iphone X
2018-05-21	51000	iphone 8

У цьому випадку для кожного рядка з таблиці orders буде виконуватися підзапит, результат якого залежить від стовпця product_id. І кожний підзапит може повертати різні дані.

Корелюючий підзапит може виконуватися і для тієї ж таблиці, до якої виконується головний запит. Наприклад, виберемо з таблиці products ті товари, вартість яких вище середньої ціни товарів для певного виробника:

```
53 •  select product_name, manufacturer, price,
54    (select avg(price) from products as subprods
55    where subprods.manufacturer=prods.manufacturer) as avgprice
56    from products as prods
57    where price>
58    (select avg(price)  from products as subprods
59    where subprods.manufacturer=prods.manufacturer)
60
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	manufacturer	price	avgprice
iphone X	Apple	76000	56333.3333
Galaxy S9	Samsung	56000	51000.0000

Тут задано два корелюючі підзапити. Перший підзапит визначає специфікацію стовпця avg(price). Він буде виконуватися для кожного рядка, що береться з таблиці products. У підзапит передається виробник товару, і на його основі вибирається

середня ціна для товарів саме цього виробника. І оскільки виробник товарів може відрізнятися, то й результат підзапиту в кожному випадку також може відрізнятися.

Другий підзапит аналогічний, тільки він використовується для фільтрації рядків із таблиці products. І також він буде виконуватися для кожного рядка.

Щоб уникнути подвійності при фільтрації в підзапиті під час порівняння виробників (SubProds.Manufacturer = Prods.Manufacturer) для зовнішньої вибірки встановлений псевдонім Prods, а для вибірки з підзапитів визначено псевдонім SubProds. Встановлення псевдонімів на таблиці є ключовою ознакою корелюючих підзапитів.

Потрібно враховувати, що оскільки корелюючі підзапити виконуються для кожного окремого рядка вибірки, то виконання таких підзапитів може уповільнювати виконання всього запиту загалом.

Підзапити в основних командах SQL.

В операторі **SELECT** можна вводити підзапити такими способами:

1. В умові виразу **WHERE**.
2. В умові виразу **HAVING**.
3. У виразі **FROM** як таблиця для вибірки.
4. У виразі **SELECT** як специфікація стовпця.

Розглянемо деякі з цих випадків. Наприклад, отримаємо всі товари, в яких вартість вище середньої:

```
53 •  select * from products
54    where price > (select avg(price) from products);
55
```

Result Grid | Filter Rows: | Edit: | Export/Import:

id	product_name	manufacturer	product_count	price
1	iphone X	Apple	2	76000
2	iphone 8	Apple	2	51000
4	Galaxy S9	Samsung	2	56000
NULL	NULL	NULL	NULL	NULL

Щоб отримати потрібні товари, треба спочатку виконати запит на отримання середньої вартості товару:

```
select avg(price) from products;
```

Оператор IN.

Нерідко підзапит використовують разом з оператором IN, який вибирає з набору значень. У підзапиті якраз може задати потрібний набір значень. Наприклад, виберемо всі товари з таблиці products, на які є замовлення в таблиці orders:

```
53 •  select * from products
54    where id in (select product_id from orders)
55
```

Result Grid | Filter Rows: | Edit: | Export/Import:

id	product_name	manufacturer	product_count	price
1	iphone X	Apple	2	76000
2	iphone 8	Apple	2	51000
5	Galaxy S8	Samsung	1	46000
NULL	NULL	NULL	NULL	NULL

Тобто підзапит у цьому випадку вибирає всі ідентифікатори товарів orders, потім за цими ідентифікаторами вибираються товари з products.

Додавши оператор **NOT**, можна вибрати ті товари, на які немає замовлень в таблиці orders:

```
53 •  select * from products
54    where id not in (select product_id from orders)
55
```

Result Grid | Filter Rows: | Edit: | Export/Import:

id	product_name	manufacturer	product_count	price
3	iphone 7	Apple	5	42000
4	Galaxy S9	Samsung	2	56000
6	Honor 10	Huawei	2	26000
7	Nokia 8	HMD Global	6	38000
NULL	NULL	NULL	NULL	NULL

Зазначимо, що це не найефективніший спосіб для вибірки зв'язаних даних з інших таблиць, оскільки для зведення даних із різних таблиць можна використовувати оператор **JOIN**, про який ітиметься нижче.

Отримання набору значень.

Під час використання в операторах порівняння підзапити повинні повертати одне скалярне значення. Але іноді виникає необхідність отримати набір значень. Щоб під час використання в операторах порівняння підзапит міг повертати набір значень, перед ним необхідно використовувати один з операторів: **ALL**, **SOME** або **ANY**.

Під час використання ключового слова **ALL** умова в операції порівняння повинна бути вірнj. для всіх значень, які повертає підзапит. Наприклад, знайдемо всі товари, вартість яких менше, ніж у будь-якого товару фірми Apple:

```
53 •  select * from products
54    where price < all(select price from products where manufacturer = 'Apple')
55
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	product_name	manufacturer	product_count	price
6	Honor 10	Huawei	2	26000
7	Nokia 8	HMD Global	6	38000
NULL	NULL	NULL	NULL	NULL

Припустимо, якщо цей під запит повертає значення val1, val2 і val3, то умова фільтрації фактично була б аналогічною до об'єднання цих значень через оператор **AND**:

```
where price < val1 and price < val2 and price < val3;
```

Водночас такий підзапит набагато простіше записати у вигляді:

```
select * from products
where price < (select min(price) from products where manufacturer-'Apple');
```

Як працює оператор **ALL**:

- x > **ALL** (1, 2) еквівалентно x > 2
- x < **ALL** (1, 2) еквівалентно x < 1
- x = **ALL** (1, 2) еквівалентно (x = 1) **AND** (x = 2)
- x <> **ALL** (1, 2) еквівалентно x **NOT IN** (1, 2).

Для операторів **ANY** і **SOME** умова операції порівняння повинна бути істинною хоча б для одного зі значень, які повертає підзапит. Дія цих операторів аналогічна тому можна використовувати будь-який із них. Наприклад, отримаємо товари, які коштують менше найдорожчого товару компанії Apple:

```
select * from products
where price < any(select price from products where manufacturer="Apple");
```

Також зауважимо, що цей запит можна зробити простішим, переписавши у вигляді:

```
select * from products
where price < (select max(price) from products where manufacturer="Apple");
```

Як працює оператор **ANY** (а також **SOME**):

- x > **ANY** (1, 2) еквівалентно x > 1
- x < **ANY** (1, 2) еквівалентно x < 2

- x = **ANY** (1, 2) еквівалентно x **NOT** (1, 2)
- x <> **ANY** (1, 2) еквівалентно (x <> 1) **OR** (x <> 2).

Підзапит як специфікація стовпця.

Результат підзапиту може бути окремим стовпцем у вибірці. Наприклад, виберемо всі замовлення і додамо до них інформацію про назву товару:

```
53 •  select *,
54    (select product_name from products where id = orders.product_id) as product
55    from orders
56
57
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	product_id	product_count	created_at	price	product
1	5	2	2018-05-21	46000	Galaxy S8
2	1	1	2018-05-23	76000	iphone X
3	2	1	2018-05-21	51000	iphone 8

Підзапити в команді INSERT. У команді **INSERT** підзапити можна використовувати для задання значення, яке вставляється в один зі стовпчиків:

```
32 •  insert into orders (product_id, created_at, product_count,  price)
33    values (
34    (select id from products where product_name='Galaxy S8'),
35    '2018-05-21',
36    2,
37    (select price from products where product_name='Galaxy S8')
38    ),
```

Підзапити в команді UPDATE.

У команді **UPDATE** використовують підзапити:

1. Для встановлення значення після оператора **SET**
2. Як частина умови у виразі **WHERE**.

Наприклад, збільшимо в таблиці orders кількість придбаних товарів компанії Apple на 2:

```
52 •   update orders
53     set product_count = product_count + 2
54     where product_id in (select id from products where manufacturer = 'Apple');
55
```

Або встановимо для замовлення вартість товару, отриману у результаті підзапиту:

```
52 •   update orders
53     set price = (select price from products where id = orders.product_id) + 3000
54     where id = 1
```

Підзапити в команді DELETE.

У команді **DELETE** підзапити використовують як частину умови. Наприклад, видалимо всі замовлення на Galaxy S8:

```
52 •   delete from orders
53     where product_id = (select id from products where product_name='Galaxy S8');
```

Оператор EXIST. Перевіряє, повертає або підзапит або будь-яке значення. Як правило, цей оператор використовують для індикації того, що як мінімум один рядок в таблиці задовольняє деякій умові. Оскільки повернення набору рядків не відбувається, то підзапити з таким оператором виконуються дуже швидко. Синтаксис оператора:

WHERE [NOT] EXISTS (підзапит)

Наприклад, знайдемо всі товари з таблиці Products, на які є замовлення в таблиці orders:

```
52 •  select * from products
53    where exists
54    (select * from orders where orders.product_id = products.id)
55
56
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	product_name	manufacturer	product_count	price
1	iphone X	Apple	2	76000
2	iphone 8	Apple	2	51000
5	Galaxy S8	Samsung	1	46000
NULL	NULL	NULL	NULL	NULL

Якщо ми хочемо дізнатися, навпаки, чи є в таблиці рядки, які НЕ задовольняють умові, то можна використати оператор **NOT EXISTS**. Наприклад, знайдемо всі товари з таблиці products, на які не було замовлень в таблиці orders:

```
52 •  select * from products
53    where not exists (select * from orders where products.id = orders.id)
```

Зауважимо, що для отримання подібного результату можна було б використати і оператор **IN**:

```
52 •  select * from products
53    where id not in (select product_id from orders)
```

Але оскільки під час застосування **EXISTS** не відбувається вибірка рядків, то його використання оптимальніше і ефективніше, ніж оператора **IN**.

Функція CASE. Перевіряє істинність набору умов і залежно від результату перевірки може повертати той чи інший результат. Пошукова функція має форму:

CASE

WHEN умова_1 THEN результат_1

WHEN умова_2 THEN результат_2

……

WHEN умова _N THEN результат_N

[ELSE альтернативний_результат]

END;

Простий формат функції **CASE** має вигляд

CASE значення

WHEN значення-1 THEN вираз-1

WHEN значення-2 THEN вираз-2

...

ELSE вираз-n

END;

Розглянемо для прикладу таку таблицю products:

```
4  create table products(
5  id int auto_increment primary key,
6  product_name varchar(30) not null,
7  manufacturer varchar(30) not null,
8  product_count int default 0,
9  price decimal not null);
```

Виконаємо запит до цієї таблиці і використаємо пошукову функцію **CASE**:

```
52   select product_name, product_count,
53  case
54    when product_count = 1
55       then 'Товар закінчується'
56    when product_count = 2
57       then 'Мало товару'
58    when product_count = 3
59       then 'Є в наявності'
60     else 'Багато товару'
61  end as category
62  from products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	product_count	category
iphone X	2	Мало товару
iphone 8	2	Мало товару
iphone 7	5	Багато товару
Galaxy S9	2	Мало товару
Galaxy S8	1	Товар закінчується
Honor 10	2	Мало товару
Nokia 8	6	Багато товару

Виконаємо запит до цієї таблиці з використанням простого формату функції **CASE**:

```
select id, product_name, case manufacturer
                              when 'Apple' then 'A'
                              when 'Huawei' then 'H'
                              else 'HMD' end as manufacturer,
        product_count, price from product;
```

id	product_name	manufacturer	product_count	price
1	iphone X	A	2	100
2	iphone 8	A	2	51834
3	iphone 7	A	5	42086
4	Galaxy S9	HMD	2	56532
5	Galaxy S8	HMD	1	46066
6	Honor 10	H	2	26565
7	Nokia 8	HMD	6	38855
8	iphone X1	A	7	56045

Операції реляційної алгебри.

Найпоширеніший підхід до з'єднання різних таблиць передбачає використання оператора **JOIN**.

Оператор INNER JOIN. Внутрішнє з'єднання таблиць. Загальний формальний синтаксис застосування оператора **INNER JOIN**:

SELECT стовпець 1, стовпець 2, …
FROM таблиця 1
[INNER] JOIN таблиця 2

ON умова 1

[[INNER] JOIN таблиця 3

ON умова 2]

Після оператора **JOIN** вказують назву другої таблиці, з якої треба додати дані до вибірки. Перед **JOIN** може бути необов'язкове ключове слово **INNER**. Його наявність або відсутність ні на що не впливає. Після ключового слова **ON** вказується умова з'єднання. Ця умова встановлює, як дві таблиці будуть порівнюватися. У більшості випадків для з'єднання застосовуються первинний ключ головної таблиці і зовнішній ключ залежної таблиці.

Для прикладу розглянемо таблиці:

```
4  create table products(
5  id int auto_increment primary key,
6  product_name varchar(30) not null,
7  manufacturer varchar(30) not null,
8  product_count int default 0,
9  price decimal not null);

11 create table customers(
12 id int auto_increment primary key,
13 first_name varchar (30) not null
14 );

19 create table orders(
20 id int auto_increment primary key,
21 product_id int not null,
22 customer_id int not null,
23 product_count int default 1,
24 created_at date not null,
25 price decimal not null,
26 foreign key (product_id) references products(id) on delete cascade,
27 foreign key (customer_id) references customers(id) on delete cascade
28 );
```

Таблиці містять певні дані:

```
insert into products (product_name, manufacturer, product_count, price)
values
('iphone X', 'Apple', 2, 76000),
('iphone 8', 'Apple', 2, 51000),
('iphone 7', 'Apple', 5, 42000),
('Galaxy S9', 'Samsung', 2, 56000),
('Galaxy S8', 'Samsung', 1, 46000),
('Honor 10', 'Huawei', 2, 26000),
('Nokia 8', 'HMD Global', 6, 38000);
--
```

```
insert into customers (first_name)
values
('Базальт'),
('Корона'),
('Галекспо'),
('Фенікс'),
('Світоч');
```

```
insert into orders (product_id, customer_id, created_at, product_count, price)
values (
(select id from products where product_name='Galaxy S8'),
(select id from customers where first_name='Фенікс'),
'2018-05-21',
2,
(select price from products where product_name='Galaxy S8')
),
(
(select id from products where product_name='iphone X'),
(select id from customers where first_name='Корона'),
'2018-05-23',
1,
(select price from products where product_name='iphone X')
),
(
(select id from products where product_name='iphone 8'),
(select id from customers where first_name='Галекспо'),
'2018-05-21',
1,
(select price from products where product_name='iphone 8')
);
```

Використовуючи **JOIN**, виберемо всі замовлення і додамо до них інформацію про товари:

```
75 •  select orders.created_at, orders.product_count, products.product_name
76    from orders
77    join products on products.id=orders.product_id;
78
79
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

created_at	product_count	product_name
2018-05-21	2	Galaxy S8
2018-05-23	1	iphone X
2018-05-21	1	iphone 8

Оскільки таблиці можуть мати стовпці з однаковими назвами, то під час вказування стовпців для вибірки вказують їх повну назву з назвою таблиці, наприклад orders.product_count.

Використовуючи псевдоніми для таблиць, можна скоротити код:

```
75 •  select o.created_at, o.product_count, p.product_name
76    from orders as o
77    join products as p
78    on p.id=o.product_id;
```

Також можна приєднувати дані відразу з декількох таблиць. Наприклад, додамо до замовлення інформацію про покупця з таблиці customers:

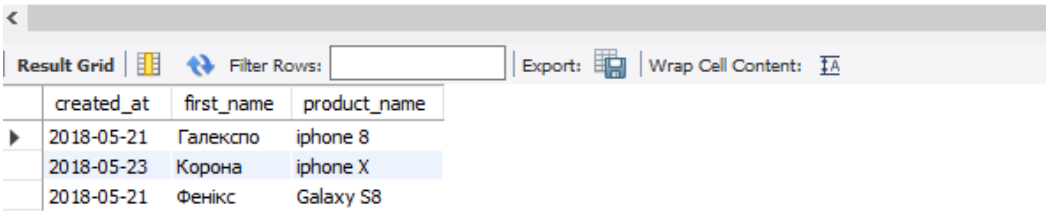
```
75 •  select orders.created_at, customers.first_name, products.product_name
76    from orders
77    join products on products.id=orders.product_id
78    join customers on customers.id=orders.customer_id;
79
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

created_at	first_name	product_name
2018-05-21	Фенікс	Galaxy S8
2018-05-23	Корона	iphone X
2018-05-21	Галекспо	iphone 8

Завдяки з’єднанню таблиць можна використовувати їх стовпці для фільтрації вибірки або її сортування:

```
75 •  select orders.created_at, customers.first_name, products.product_name
76    from orders
77    join products on products.id=orders.product_id
78    join customers on customers.id=orders.customer_id
79    where products.price > 45000
80    order by customers.first_name;
```

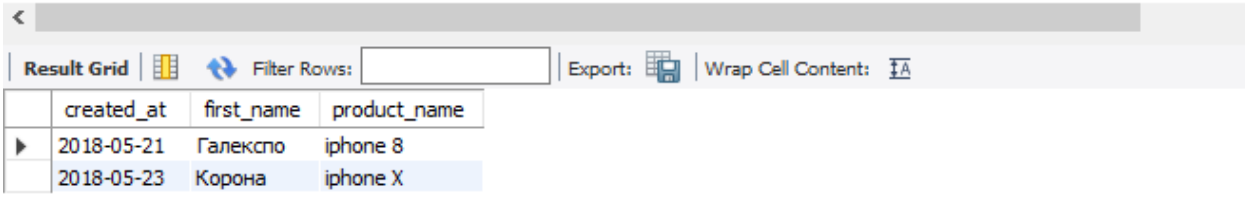


Result Grid | Filter Rows: | Export: | Wrap Cell Content:

created_at	first_name	product_name
2018-05-21	Галекспо	iphone 8
2018-05-23	Корона	iphone X
2018-05-21	Фенікс	Galaxy S8

Умови після ключового слова **ON** можуть бути складнішими за складом:

```
75 •  select orders.created_at, customers.first_name, products.product_name
76    from orders
77    join products on products.id=orders.product_id and products.manufacturer='Apple'
78    join customers on customers.id=orders.customer_id
79    where products.price > 45000
80    order by customers.first_name;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

created_at	first_name	product_name
2018-05-21	Галекспо	iphone 8
2018-05-23	Корона	iphone X

У цьому випадку вибираємо всі замовлення на товари, виробником яких є Apple.

Під час використання оператора **JOIN** потрібно враховувати, що процес з'єднання таблиць може бути ресурсоємним, тому слід з'єднувати тільки ті таблиці, дані з яких дійсно необхідні. Чим більше таблиць з'єднується, тим більше знижується продуктивність.

Оператор OUTER JOIN. Зовнішнє з'єднання таблиць. На відміну від Inner Join зовнішнє з'єднання повертає всі рядки однієї або двох таблиць, які беруть участь у з'єднанні. Формальний синтаксис Outer Join:

SELECT стовпці

FROM таблиця 1

{LEFT|RIGHT} [OUTER] JOIN таблиця 2 ON умова 1

[{LEFT|RIGHT} [OUTER] JOIN таблиця 3 ON умова 2]…

Перед оператором **JOIN** вказують одне з ключових слів **LEFT** або **RIGHT**, які визначають тип з'єднання:

- **LEFT**: вибірка буде містити всі рядки з першої або лівої таблиці
- **RIGHT**: вибірка буде містити всі рядки з другої або правої таблиці.

Також перед оператором **JOIN** можна вказати ключове слово **OUTER**, але його використання необов'язкове. Після **JOIN** вказують таблицю, яку треба приєднати, а далі – умову з'єднання.

Наприклад, з'єднаємо таблиці orders і customers:

```
75 •  select first_name, created_at, product_count, price, product_id
76    from orders
77    left join customers on orders.customer_id=customers.id;
78
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

first_name	created_at	product_count	price	product_id
Фенікс	2018-05-21	2	46000	5
Корона	2018-05-23	1	76000	1
Галекспо	2018-05-21	1	51000	2

Таблиця orders є першою або лівою таблицею, а таблиця customers – правою таблицею. Тому, оскільки тут використовується вибірка за лівою таблицею, то спочатку будуть вибиратись рядки з orders, а потім до них за умовою orders.customer_id = customers.od будуть додаватися зв'язані рядки з customers.

Left Join вибирає всі рядки першої таблиці і потім приєднує до них рядки правої таблиці. Для прикладу, візьмемо таблицю customers і додамо до покупців інформацію про їхні замовлення.

У випадку з **LEFT JOIN** MySQL вибирає спочатку всіх покупців з таблиці customers, потім співставляє їх з замовленнями з таблиці orders через умову orders.customer_id = customers.id. Однак не у всіх покупців є замовлення, тоді покупцю для відповідних стовпців встановлюються значення **NULL**.

```
72    #inner join
73 •  select first_name, created_at, product_count, price
74    from customers join orders
75    on orders.customer_id = customers.id;
76
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

first_name	created_at	product_count	price
Фенікс	2018-05-21	2	46000
Корона	2018-05-23	1	76000
Галекспо	2018-05-21	1	51000

```
72    #left join
73 •  select first_name, created_at, product_count, price
74    from customers left join orders
75    on orders.customer_id = customers.id;
76
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

first_name	created_at	product_count	price
Базальт	NULL	NULL	NULL
Корона	2018-05-23	1	76000
Галекспо	2018-05-21	1	51000
Фенікс	2018-05-21	2	46000
Світоч	NULL	NULL	NULL

Змінимо у прикладі вище тип з’єднання для **OUTER JOIN** з лівостороннього на правосторонній.

Тепер будуть вибиратися всі рядки з orders (з правої таблиці), а до них уже будуть приєднуватись зв'язані за умовою рядки з таблиці customers:

```
72    #right join
73 •  select first_name, created_at, product_count, price
74    from customers right join orders
75    on orders.customer_id = customers.id;
76
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

first_name	created_at	product_count	price
Базальт	NULL	NULL	NULL
Корона	2018-05-23	1	76000
Галекспо	2018-05-21	1	51000
Фенікс	2018-05-21	2	46000
Світоч	NULL	NULL	NULL

Використовуємо лівостороннє з’єднання для додавання до замовлень інформацію про покупців і товари:

```
72 •  select customers.first_name, orders.created_at, products.product_count, products.manufacturer
73    from orders
74    left join customers on orders.customer_id = customers.id
75    left join products on orders.product_id = products.id;
76
77
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

first_name	created_at	product_count	manufacturer
Фенікс	2018-05-21	1	Samsung
Корона	2018-05-23	2	Apple
Галекспо	2018-05-21	2	Apple

Також можна застосовувати комплексні умови з фільтрацією і сортуванням. Наприклад, виберемо всі замовлення з інформацією про клієнтів і товари за тими товарами, у яких вартість менше 45000, і відсортуємо їх за датою замовлення:

```
72 •  select customers.first_name, orders.created_at, products.product_count, products.manufacturer
73    from orders
74    left join customers on orders.customer_id = customers.id
75    left join products on orders.product_id = products.id
76    where products.price > 45000
77    order by orders.created_at;
78
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

first_name	created_at	product_count	manufacturer
Фенікс	2018-05-21	1	Samsung
Галекспо	2018-05-21	2	Apple
Корона	2018-05-23	2	Apple

Або виберемо всіх покупців з customers, у кого немає замовлень в таблиці orders:

```
72 •  select first_name from customers
73    left join orders on customers.id = orders.customer_id
74    where customer_id is null;
```

Також можна комбінувати INNER JOIN і **OUTER JOIN**:

```
72 •  select customers.first_name, orders.created_at, products.product_name, products.manufacturer
73    from orders
74    join products on orders.product_id = products.id and products.price > 45000
75    left join customers on orders.customer_id = customers.id
76    order by orders.created_at;
```

Спочатку за умови до таблиці orders через **INNER JOIN** приєднається зв’язана інформація з products, потім через **OUTER JOIN** додається інформація з таблиці customers.

Виконання теоретико-множинних операцій реляційної алгебри засобами SQL

Оператор UNION. Дає змогу об’єднати дві однотипні вибірки. Ці вибірки можуть бути з різних таблиць або з однієї і тієї ж таблиці. Формальний синтаксис об’єднання:

SELECT вираз_1

UNIOIN [ALL] SELECT вираз_2

[UNION [ALL] SELECT вираз_N]

Наприклад, нехай в базі даних будуть дві окремі таблиці для клієнтів банку (таблиця customers) і для працівників банку (таблиця employees):

```
create schema bank;
create table customers
(
id int auto_increment primary key,
first_name varchar(20) not null,
last_name varchar(20) not null,
account_sum decimal);

create table employees
(
id int auto_increment primary key,
first_name varchar(20) not null,
last_name varchar(20) not null);

insert into customers(first_name, last_name, account_sum)
values
('Tom', 'Smith', 2000),
('Sem', 'Brown', 3000),
('Mark', 'Adams', 2500),
('Paul', 'Ins', 4200),
('John', 'Smith', 2800),
('Tim', 'Cook', 2800);

insert into employees(first_name, last_name)
values
('Homer', 'Simpson'),
('Tom', 'Smith'),
('Mark', 'Adams'),
('Nik', 'Svensson');
```

Зауважимо, що обидві таблиці, незважаючи на наявність різних даних, можна характеризувати за двома спільними атрибутами – ім’ям (first_name) і прізвищем

(last_name). Виберемо відразу всіх клієнтів банку і його співробітників з обох таблиць Під час об’єднання кількість та тип стовпців, які вибираються, співпадає:

```
32 •   select first_name, last_name
33     from customers
34     union select first_name, last_name from employees;
35
```

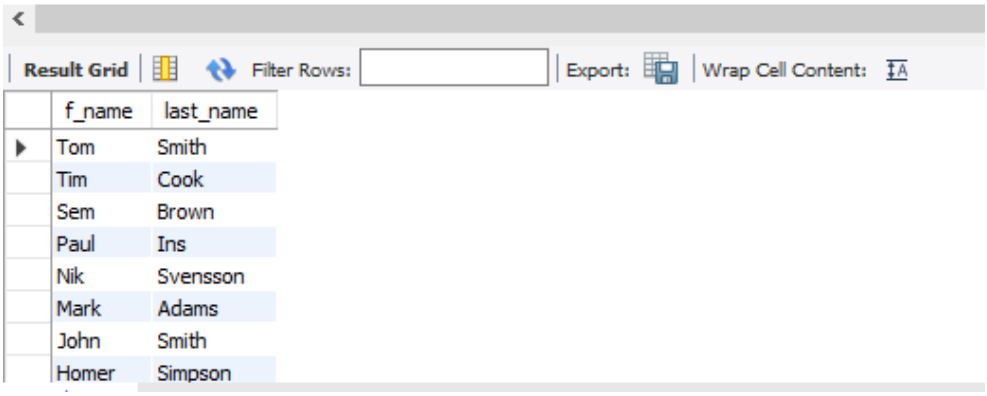
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

first_name	last_name
Tom	Smith
Sem	Brown
Mark	Adams
Paul	Ins
John	Smith
Tim	Cook
Homer	Simpson
Nik	Svensson

При цьому назви стовпців об’єднаної вибірки будуть співпадати з назвами стовпців першої вибірки. І якщо ми хочемо провести сортування, то у виразах **ORDER BY** необхідно орієнтуватися саме на назви стовпців першої вибірки.

У цьому випадку кожна вибірка має відповідати стовпцю f_name з першої вибірки. Однак під час сортування буде враховуватись і значення стовпця first_name з другої вибірки:

```
32 •   select first_name as f_name, last_name
33     from customers
34     union select first_name, last_name
35     from employees
36     order by f_name desc;
37
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

f_name	last_name
Tom	Smith
Tim	Cook
Sem	Brown
Paul	Ins
Nik	Svensson
Mark	Adams
John	Smith
Homer	Simpson

Якщо в одній вибірці більше стовпців, ніж в іншій, то їх не можна об’єднати. Наприклад, у наступному випадку об’єднання завершиться з помилкою:

```
32 •   select first_name,last_name, account_sum
33     from customers
34     union select first_name, last_name
35     from employees;
```

Якщо обидва набори, які об’єднують, містять в рядках ідентичні значення, то під час об’єднання повторювані рядки видаляються. Наприклад, у випадку з таблицями customers і employees співробітники банку можуть бути одночасно його клієнтами і міститися в обох таблицях. Під час об’єднання в прикладах вище всі дубльовані рядки видалялися. Якщо ж необхідно під час об'єднання зберегти все, в тому числі повторювані рядки, то для цього необхідно використовувати оператор **ALL**:

```
32 •  select first_name,last_name
33    from customers
34    union all select first_name, last_name
35    from employees
36    order by first_name;
37
38
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

first_name	last_name
Homer	Simpson
John	Smith
Mark	Adams
Mark	Adams
Nik	Svensson
Paul	Ins
Sem	Brown
Tim	Cook
Tom	Smith
Tom	Smith

Об’єднувати вибірки можна і з однієї і тієї ж таблиці. Наприклад, залежно від суми на рахунку клієнта нам потрібно нараховувати йому певні відсотки. У цьому випадку, якщо сума менше 3000, то нараховуються відсотки у розмірі 10% від суми на рахунку. Якщо на рахунку більше 3000, то відсотки збільшуються до 30%.

```
32 •  select first_name,last_name, account_sum + account_sum * 0.1 as total_sum
33    from customers where account_sum < 3000
34    union select first_name, last_name, account_sum + account_sum * 0.3 as total_sum
35    from customers where account_sum >= 3000;
36
37
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

first_name	last_name	total_sum
Tom	Smith	2200.0
Mark	Adams	2750.0
John	Smith	3080.0
Tim	Cook	3080.0
Sem	Brown	3900.0
Paul	Ins	5460.0

Оператор INTERSECT використовують для написання запиту на переріз двох сумісних за типом реляційних відношень. Обов'язковою умовою є співпадіння заголовків відношень:

SELECT * FROM name_table_A INTERSECT SELECT * FROM name_table_B

Однак не у всіх версіях MySQL цей оператор присутній. Тоді можна використати інші оператори вибірки, додаючи додаткові умови вибору даних з таблиць.

Для прикладу створимо базу даних fabryka, яка містить таблиці зі списками виробників list_first, list_second.

```
1   create schema fabryka;
2   create table list_first
3   (
4   id int auto_increment primary key,
5   first_name varchar(20) not null,
6   account_sum decimal);
7
8   create table list_second
9   (
10  id int auto_increment primary key,
11  first_name varchar(20) not null,
12  account_sum decimal);
```

```
14 •  insert into list_first(first_name, account_sum)
15    values
16    ('Мрія', 1500),
17    ('Диво', 2400),
18    ('Каприз', 1450),
19    ('Берізка', 2510);
20
21 •  insert into list_second(first_name, account_sum)
22    values
23    ('Диво', 2400),
24    ('Каприз', 1450),
25    ('Калина', 1250),
26    ('Дружба', 2100);
```

Один із варіантів коду отримання перерізу цих таблиць представлено нижче:

```
28 •  select * from list_first where exists
29    (select * from list_second where list_first.first_name = first_name)
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	first_name	account_sum
2	Диво	2400
3	Каприз	1450
NULL	NULL	NULL

Якщо переріз потрібно отримати для підмножин записів, які вибирають з однієї таблиці з простим ключем, можна використовувати умову **IN**. Нехай, наприклад, в таблиці A визначений ключ id. Наведений нижче фрагмент запису дає змогу отримати множину рядків, які задовольняють і умові запиту, і додатковим умовам зовнішнього запиту (фактично – це переріз):

SELECT * FROM name_table_A
WHERE id IN(SELECT id FROM name_table_A AS name_table_B WHERE…)

Оператор EXCEPT для виконання запиту на різницю двох сумісних за типом реляційних відношень. Обов'язковою умовою є співпадіння заголовків відношень:

SELECT * FROM name_table_A EXCEPT SELECT * FROM name_table_B

Якщо потрібно отримати різницю між двома підмножинами рядків з однієї таблиці з простим ключем, для запису різниці можна використати умову **NOT IN** у розділі **WHERE**.

Наприклад, різницю відношень list_first та list_second можна за допомогою такого коду:

```
28 •   select * from list_first
29     where first_name not in (select first_name from list_second);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	first_name	account_sum
1	Мрія	1500
4	Берізка	2510
NULL	NULL	NULL

Для виконання **декартового добутку** на SQL в синтаксисі ANSI SQL-92 команда буде мати вигляд:

SELECT * from name_table_A CROSS JOIN name_table_B;

Альтернативний варіант запису:

SELECT * FROM name_table_A, name_table_B;

Для ілюстрації виконання команди створимо дві таблиці worker (працівник) та money (квартальна премія) та внесемо в них деяку інформацію:

```
28  create table worker
29  (
30  id int auto_increment primary key,
31  first_name varchar(20) not null,
32  account_sum decimal);
33
34  create table money
35  (
36  id int auto_increment primary key,
37  account_sum decimal);
```

```
39  insert into worker(first_name, account_sum)
40  values
41  ('Козак', 2400),
42  ('Коваль', 1450),
43  ('Супрун', 1250),
44  ('Яценко', 2100);
45
46  insert into money(account_sum)
47  values
48  (1500),
49  (1700);
```

Результатом виконання декартового добутку утворених таблиць за альтернативним записом буде таблиця:

```
51  select * from worker, money;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	first_name	account_sum	id	account_sum
1	Козак	2400	2	1700
1	Козак	2400	1	1500
2	Коваль	1450	2	1700
2	Коваль	1450	1	1500
3	Супрун	1250	2	1700
3	Супрун	1250	1	1500
4	Яценко	2100	2	1700
4	Яценко	2100	1	1500

Розглянемо запис **операції проєкції**. Проєкція відношення A за атрибутами V, W, Z в реляційній алгебрі позначається як A [V, W, Z]. Під час запису запиту на

проєкцію на SQL необхідно враховувати, що результат може мати рядки, які повторюються. Для отримання точної відповідності запиту і реляційної операції потрібно використати інструкцію **DISTINCT**

SELECT DISTINCT V, W, Z FROM name_table_A;

На прикладі таблиці products.

```
5  create table products(
6  id int auto_increment primary key,
7  product_name varchar(30) not null,
8  manufacturer varchar(30) not null,
9  product_count int default 0,
10 price decimal not null);
```

```
32 insert into products (product_name, manufacturer, product_count, price)
33 values
34 ('iphone X', 'Apple', 2, 76000),
35 ('iphone 8', 'Apple', 2, 51000),
36 ('iphone 7', 'Apple', 5, 42000),
37 ('Galaxy S9', 'Samsung', 2, 56000),
38 ('Galaxy S8', 'Samsung', 1, 46000),
39 ('Honor 10', 'Huawei', 2, 26000),
40 ('Nokia 8', 'HMD Global', 6, 38000);
```

Побудуємо проекцію таблиці, вибравши поля з назвою продукції та ціною:

```
79 select distinct id, product_name, price from products;
80
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Conten

id	product_name	price
1	iphone X	76000
2	iphone 8	51000
3	iphone 7	42000
4	Galaxy S9	56000
5	Galaxy S8	46000
6	Honor 10	26000
7	Nokia 8	38000
NULL	NULL	NULL

Для створення **проєкції** на мові SQL можна використовувати директиву створення віртуальних таблиць **CREATE VIEW**:

CREATE VIEW ім'я_проєкції [(перелік_полів)]
AS SELECT DISTINCT (перелік_полів) FROM ім'я_таблиці

Розділення таблиць на частини.

Подібно до нарізання піци перед тим, як її з'їсти, розділення таблиці – це процес розбиття величезних таблиць на менші, щоб ви могли легше та ефективніше шукати дані.

Вертикальне та горизонтальне розділення.

Існують два типи розділення таблиць – вертикальне та горизонтальне.

Вертикальне розділення. Вертикальне розділення передбачає спочатку створення таблиць із меншою кількістю стовпців, а потім використання додаткових таблиць для розміщення решти стовпців. Оскільки рядки розбиваються відповідно до їхніх стовпців, вертикальний поділ також відомий як поділ рядків. На даний момент MySQL не підтримує вертикальне розбиття у своїй базі даних.

Горизонтальне розділення. Логіка розділення розбиває рядки на кілька таблиць. Кількість стовпців залишається незмінною в усіх розділах, а кількість рядків може змінюватися.

До основних типів горизонтального розділення таблиць, які доступні з MySQL 5.7, належать: **RANGE**, **LIST**, **HASH (LINEAR HASH)**, **KEY (LINEAR KEY)**.

Оператор RANGE. Розділення на діапазони відбувається таким чином, що кожен розділ містить рядки, для яких значення виразу розділення потрапляє в заданий діапазон. Діапазони мають бути неперервними, але не перекриватися, і визначаються за допомогою оператора **VALUES LESS THAN**.

Увага!!! Під час розділення таблиці на кілька частин за будь-яким стовпцем, може виникати помилка, якщо один зі стовпців таблиці оголошений як **PRIMARY KEY** або **UNIQUE**.

Розглянемо приклад. Припустимо, що ви створюєте таблицю, подібну до наведеної нижче, для зберігання особистої інформації для мережі з 20 магазинів, пронумерованих від 1 до 20:

```
create table employee (
    id int not null,
    fname varchar(30),
    lname varchar(30),
    hired date not null default '1970-01-01',
    separated date not null default '9999-12-31',
    job_code int not null,
    store_id int not null
);
```

Заповнимо таблицю даними:

```
insert into employee(id, fname, lname, job_code, store_id)
values
(1,'Piotr', 'Konja', 5, 1),
(2,'Konrad', 'List', 3, 1),
(3,'Jan', 'Lasota', 5, 2),
(4,'Kamil', 'Dubak', 6, 3),
(5,'Katarina', 'Kowalchyk', 6, 3),
(6,'Ilona', 'Zdeba', 3, 5),
(7,'Elena', 'Swecz', 3, 6),
(8,'Olga', 'Mala', 9, 8),
(9,'Adam', 'Marenski', 9, 12),
(10,'Jakub', 'Kowal', 9, 15),
(11,'Michal', 'Kryshtal', 6, 8),
(12,'Dariush', 'Zych', 6, 4),
(13,'Piotr', 'Trata', 10, 16),
(14,'Maksymilian', 'Kowalski', 5, 18),
(15,'Viktor', 'Sawicki', 10, 13),
(16,'Anna', 'Pasha', 9, 10);
```

Цю таблицю можна розділити різними способами, залежно від ваших потреб. Одним зі способів може бути використання стовпця код_магазину (store_id).

Наприклад, ви можете вирішити розділити таблицю 4 способами, додавши пропозицію **PARTITION BY RANGE (назва_стовпця)**, як показано нижче:

```
alter table employee partition by range (store_id) (
    partition p0 values less than (6),
    partition p1 values less than (11),
    partition p2 values less than (16),
    partition p3 values less than maxvalue
);
```

MAXVALUE представляє ціле значення, яке завжди більше за найбільше можливе ціле значення (на математичній мові воно слугує мінімумом верхньої межі). Тепер ви можете шукати дані з будь-якої створеної частини таблиці:

```
40  select * from employee partition (p1)
41  where store_id between 7 and 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	fname	lname	hired	separated	job_code	store_id
8	Olga	Mala	1970-01-01	9999-12-31	9	8
11	Michal	Kryshtal	1970-01-01	9999-12-31	6	8
16	Anna	Pasha	1970-01-01	9999-12-31	9	10

```
40  select * from employee partition (p3)
41  where store_id between 11 and 16;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	fname	lname	hired	separated	job_code	store_id
13	Piotr	Trata	1970-01-01	9999-12-31	10	16

Розбиття таблиці може значно прискорити виконання запиту (особливо для таблиць із тисячами рядків). Інший варіант поділу **RANGE COLUMNS** дає змогу вказати більше одного стовпця розділення. Створимо розділи таблиці, які містять діапазон job_code в деякому store_id:

```
alter table employee partition by range columns (job_code, store_id) (
    partition from_5 values less than (5, 5),
    partition from_8 values less than (8, 10),
    partition from_10 values less than (10, 16),
    partition from_latest_highest values less than (maxvalue, maxvalue)
);
```

Якщо ви хочете вибрати всі рядки з кодами роботи між 5 і 8, де код магазину міститься між 7 і 9, ви можете виконати такий запит:

```
50    select * from employee partition(from_8)
51    where store_id between 7 and 9;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	fname	lname	hired	separated	job_code	store_id
11	Michal	Kryshtal	1970-01-01	9999-12-31	6	8

Оператор LIST. Розбиття **LIST** подібне до розбиття **RANGE**. Як і у випадку з розділенням **RANGE**, кожний розділ (частина) має бути визначений явно. Основна відмінність між двома типами поділу полягає в тому, що в поділі **LIST** кожний розділ визначається та вибирається на основі значення стовпця, що належить одному значенню з набору значень, а не одному значенню з набору безперервних діапазонів значень. Вказати стовпець, за яким виконуватиметься розподіл можна за допомогою

PARTITION BY LIST (expr),

де **expr** – це значення стовпця або вираз на основі значення стовпця, який повертає ціле значення.

Далі потрібно задати кожний розділ за допомогою

VALUES IN (value_list),

де **value_list** – список цілих значень, розділених комами.

Використаємо попередній приклад. Припустимо, що є 20 магазинів, розподілених між 4 франшизами, як показано в таблиці нижче:

Регіон	Ідентифікаційні номери магазинів
Північ	3, 5, 6, 9, 17
Схід	1, 2, 10, 11, 19, 20
Захід	4, 12, 13, 14, 18
Центральний	7, 8, 15, 16

Для поділу таблиці ***employee*** так, щоб рядки для магазинів, які належать одному регіону, зберігалися в одному розділі задаємо команду:

```
alter table employees partition by list(store_id) (
    partition  pNorth values in (3,5,6,9,17),
    partition  pEast values in (1,2,10,11,19,20),
    partition  pWest values in (4,12,13,14,18),
    partition  pCentral values in (7,8,15,16)
);
```

Тепер ми можна писати запити, використовуючи щойно створені розділи:

```
select * from employee partition(pEst) where (stor_id=10);
```

Оператор HASH. Поділ за допомогою **HASH** в основному використовується для забезпечення рівномірного розподілу даних між заздалегідь визначеною кількістю розділів. Розбиваючи **RANGE** або **LIST**, ви повинні явно вказати, в якому розділі має зберігатися дане значення стовпця або набір значень стовпця; для **HASH**-розбиття MySQL подбає про це за вас, і вам потрібно лише надати значення стовпця або вираз на основі значення стовпця, який буде хешуватися, і кількість розділів, на які буде розділена таблиця.

Інструкція нижче розділяє таблицю за стовпцем store_id на 4 частини:

```
create table employee (
    id int not null,
    fname varchar(30),
    lname varchar(30),
    hired date not null default '1970-01-01',
    separated date not null default '9999-12-31',
    job_code int not null,
    store_id int not null
);
alter table employee partition by hash(store_id) partitions 4;
```

Якщо не вказати у **PARTITIONS** кількість розділів, то за замовчуванням розбиття не відбувається.

Отже, розділи пришвидшують пошук. У невеликих таблицях важко побачити користь розділення на частини. Тому, якщо ваші запити виконуються повільно, а таблиця бази даних не містить мільйонів рядків, вам слід розглянути інші методи оптимізації, перш ніж розділяти таблиці.

3. Типові контрольні запитання

1. За допомогою яких операторів формують підзапити в SELECT?
2. Назвіть оператори для отримання набору значень.
3. Які функції підзапитів у командах UPDATE, INSERT, DELETE?
4. Для чого використовують оператор CASE?
5. Які оператори використовують для з’єднання таблиць?
6. Які запити виконують за допомогою операторів UNION, INTERSET, EXEPT?
7. Запишіть запит на декартовий добуток відношень A та B.
8. Які типи розділення таблиці на частини пропонує MySQL?
9. Охарактеризуйте тип розбиття таблиці RANGE.
10. Охарактеризуйте тип розбиття таблиці LIST.
11. Охарактеризуйте тип розбиття таблиці HASH.

4. Завдання для виконання лабораторної роботи

До БД обраної предметної області написати та виконати по **5 скриптів** до завдань 1 – 4:

Завдання 1. Написати **різні типи підзапитів (корелюючі та некорелюючі)** (**разом не менше 5**) в основних командах SQL до таблиць бази даних.

Завдання 2. Використовуючи оператори **INNER JOIN** та **OUTER JOIN**, написати скрипти для формування зведених таблиць з вибраних даних під час створення запитів до бази даних.

Завдання 3. Використовуючи функцію **CASE** разом із агрегатними функціями або без них, написати скрипти для створення запитів різного типу до бази даних (**не менше 3**).

Завдання 4. Використовуючи оператори **UNION**, **INTERSECT**, **EXCEPT**, написати скрипти для реалізації теоретико-множинних операцій реляційної алгебри в запитах вибірки даних з таблиць бази даних.

Завдання 5. Виконайте розділення принаймні **2 таблиць** розробленої бази даних за допомогою **RANG**, **LIST** і **HASH**.

Зміст звіту:

1. Титульний лист
2. Завдання 1.
3. Представити структуру скриптів запитів, написаних до бази даних обраної предметної області (скріншоти), та скріншоти результатів їх виконання згідно з завданням 1 лабораторної роботи.
4. Завдання 2….
5. ……
6. Висновки.

5. Список літератури

1. Мулеса О. Ю. Інформаційні системи та реляційні бази даних. Ужгород, 2018. 118 с.

2. Павельчак А. Г., Самотий В. В., Дзелендзяк У. Ю. Бази даних та знань. Львів, 2011. 115 с.

3. Вступ у MySQL. https://metanit.com/sql/mysql/2.5.php

4. https://coderlessons.com/tutorials/bazy-dannykh/osnovy-sql/7-operator-select-2

5. https://dev.mysql.com/doc/workbench/en/wb-sql-editor-query-panel.html

6. MySQL 8.0 Reference Manual / Partitioning / Partitioning Types. https://dev.mysql.com/doc/refman/8.0/en/partitioning-types.html